MDynaMix Package
version 5.3.0
User manual

Alexander Lyubartsev
Aatto Laaksonen
Alexei Nikitin

Division of Physical Chemistry
Department of Materials and Environmental Chemistry
Stockholm University

18 December 2021

Contents

1 Introduction **4**
- 1.1 Changes from previous versions . . . 4

2 Description of the molecular structure and the force field **6**
- 2.1 The force field . . . 6
- 2.2 Molecular geometry and non-bonded interaction parameters . . . 7
- 2.3 Bonds . . . 7
- 2.4 Covalent angles . . . 8
- 2.5 Dihedral angles . . . 9
- 2.6 Optional features of the force field . . . 9

3 The main input file **11**
- 3.1 General notes . . . 11
- 3.2 Basic setup . . . 11
- 3.3 Restart control . . . 12
- 3.4 System . . . 13
- 3.5 Ensemble . . . 15
- 3.6 Molecular Dynamics . . . 17
- 3.7 Startup . . . 18
- 3.8 Expanded Ensemble mode . . . 20
- 3.9 Path Integral mode . . . 23
- 3.10 Properties . . . 23

4 Compilation **25**
- 4.1 Sequential execution . . . 25
- 4.2 Parallel execution . . . 25
- 4.3 Other makefiles . . . 26

5 Execution **26**
- 5.1 Files used by the program . . . 26
- 5.2 Memory considerations . . . 26
- 5.3 Program structure . . . 27

6 *Makemol* utility **28**
- 6.1 General organization . . . 28
- 6.2 The force field (.ff) file . . . 29
- 6.3 char2mdx . . . 30
- 6.4 The molecular structure (.smol) file . . . 31

7 *Tranal* utility **31**
- 7.1 General organization . . . 31
- 7.2 tranal_base: reading trajectories . . . 33
- 7.3 Computation of the electron density and electrostatic potential in bilayers: bileldens.f . . . 35
- 7.4 Dielectric constant: diel.f . . . 36

7.5 Diffusion: diffus.f 37
7.6 Dryrun: dryrun.f 38
7.7 Hydrogen bonds: h-bonds.f 38
7.8 Lateral diffusion: latdiff.f 39
7.9 Order parameters: order.f 40
7.10 Radial distribution functions: rdf.f 41
7.11 Gyration radius and end-to-end distance: Rg.f 42
7.12 Spatial distribution functions: sdf.f 43
7.13 Distribution of torsion angles: torsion.f 45
7.14 Ramachandran plot for a pair of torsion angles: torsion2.f 46
7.15 Residence time calculation: restime.f 46
7.16 Time correlation functions: trtcf.f 47

1 Introduction

MDynaMix is a general purpose molecular dynamics code for simulations of mixtures of rigid or flexible molecules, interacting by standard molecular-mechanics force field including electrostatic and Lennard-Jones potentials for electrostatic interactions as well as terms describing covalent bonds, angles and dihedral angles. The program is well suited for simulations of flexible molecules based on the double time step algorithm. Alternatively, rigid bonds can be treated by the SHAKE algorithm. Algorithms for NVE, NVT, NPT and anisotropic NPT ensembles are implemented. The long-range electrostatic interactions are treated by the Ewald summation method, or alternatively by a reation field or simple cutoff. The program can be run both in sequential and parallel execution, in the latter case the MPI parallel environment is required. Possibility for taking into account quantum motion of nuclei using Path Integral approach is implemented. Expanded ensemble mode of simulations for calculation of solvation free energies is added from v.5.2.

The program is written in standard Fortran 77/90 and thus can be run on any computer system having a Fortran compiler.

The program is originally based on the MOLDYN program by Aatto Laaksonen, available from the CCP5 program library, Daresbury Lab, UK. Since 1993 many additional changes were made by Alexander Lyubartsev. The first parallelized version of the program (v.4.0) was published in:

A.P.Lyubartsev, A.Laaksonen, "*MDynaMix - a scalable portable parallel MD simulation package for arbitrary molecular mixtures*" Computer Physics Communications, v.128(3), pp.565-589 (2000)

Apart the main MDynaMix block, the package includes the following components:

- **makemol** utility to create molecular description files (.mmol)
- **charm2mdx** convert Charmm parameter file into MDynaMix force field parameter file **cifo** utility to convert .cif (crystollagraphic informational files) to MDynaMix molecular description files **tranal** - a suite of utilities for analyzing trajectories

1.1 Changes from previous versions

The changes made from 5.2.0 to 5.3.0

- v 5.3.1 Rg utility in the tranal suite to compute gyrationradius and end-to-end distance

  Tranal utility accept .itp files in Gromacs format describing individual molecules

- v. 5.3.0 A new version of the expanded ensemble mode implementing L2MC algorithm: A.Nikitin, V.Milchevskaya, A.P.Lyubartsev "To the fast

calculation of the solvation free energy. Combining expanded ensembles with L2MC", J. Comput. Chem., 42, 787 (2021)

Additional combination rule for Lennard-Jones interations from paper: A. Nikitin, Y. Milchevskiy, A. P. Lyubartsev "AMBER-ii: New Combining Rules and Force Field for Perfluoroalkanes" J. Phys. Chem. B, 119, 14563-14573 (2015) DOI: 10.1021/acs.jpcb.5b07233

- v. 5.2.8:

  The main input file name should be given as an argument to the "md" or "mdp" executable both in serial and parallel execution

  Flexible SPC water model accept internal bond parameters from the .mmol file

  Control of large temperature deviations became optional

- v. 5.2.7: The pair list describing special Lennard-Jones interactions (owerwriting combination rules) can be automatically generated from the force field file (.ff) and atom types present in the molecular description files (.mmol).

  Extentions to the force field file format are added

  Support for TRIPOS .mol2 file format (as produced by `Avogadro` molecular editor and the `antechamber` package) in the `makemol` utility is added.

  Changed interface to the `makemol` utility

- v. 5.2.6: Module `cifo`, converting crystallografic information in .cif format to MDynaMix .mmol and .smol molecular description formats, is added (contribution by Dmitrij Lioubartsev).

  Option for special Lennard-Jones parameters for 1-4 neighbours in a molecule is introduced (parameter par14 in .mmol file). Makemol utility is updated to use this option.

  Special Lennard-Jones interactions between any atom pairs (different from the parameters of .mmol file and what is determined by the combination rules) can be also introduced using "pair list" option of the input file.

  Option for on-the-flight computation of radial distribution functions is depriciated

- v. 5.2.5: Soft core potentials are implemented in computations of solvation free energies by the expanded ensemble method, see: J.M.P.Jämbeck,F.Mocci, A.P.Lyubartsev, A.Laaksonen, "*Partial atomic charges and their impact on the free energy of solvation*". Journal of Computational Chemistry, 34, 187-197 (2013), Supporting Information.

  Option for "gravitation"-like force acting along Z-axis on molecules of a certain type is added

  Option for on-the-flight computation of time correlation functions is depriciated

- v.5.2.4: Algorithm for WL iterations is changed. One WL interations is no longer linked to parameter `Serie_average` , but is determined by requirement that the system has walked at least `Num_pass` (default 2) times between the end subensembles.

  Module restime.f , computing residense times, is added to the analysis suite.

  .mmol file are no longer necessary for some types of analysis of trajectories.

- v.5.2.3: A bug corrected which lead to incorrect computations of solvation free energies for large molecules due to incorrect computations of intramolecular nonbonded electrostatic interaction (affect versions 5.2.1-5.2.2)

- v.5.2.2: In the expanded ensemble mode, an automatic tuning of balancing factors using Wang-Landau algorithm is implemented (contribution by Nikolai Volkov)

- v.5.2.1: Option for semianizotropic NPT ensemble, with separate pressure control along Z-axis and in XY-plane, is introduced

The changes made from v.5.1 to 5.2:

- Expanded ensemble method for calculation of solvation free energy is now included into main MDynaMix package. The program computes the free energy difference upon addition of one or several molecules. Automatic tuning of balancing factors using Wang-Landau algorithm is implemented

- Option for "semiisotropical" NPT ensemble with separate pressure conrol in Z direction and in XY plane is introduced

- Option for gradual change of volume (density) to a target value

2 Description of the molecular structure and the force field

2.1 The force field

The general form of the force field implemented in the program is:

$$U = U_{LJ} + U_{el} + U_{bond} + U_{ang} + U_{tors} + U_{impr} \tag{1}$$

where

$$U_{LJ} = \sum_{non-bonded} 4\varepsilon_{ij}\Big(\big(\frac{\sigma_{ij}}{r_{ij}}\big)^{12} - \big(\frac{\sigma_{ij}}{r_{ij}}\big)^{6}\Big) \tag{2}$$

is the sum of Lennard-Jones interactions taken over non-bonded atom pairs. A pair of atoms is considered as non-bonded if they are on different molecules or if

they are on the same molecule but separated by more than two covalent bonds. A case when a pair of atoms is separated by exactly 3 bonds (the so-called 1-4 neighbors) is treated separately, see below. r_{ij} is the distance between atoms i and j. Parameters ε_{ij} and $sigma_{ij}$ for atom pair (i, j) are determined by default from the individual ε_i and σ_i parameters for atoms using combination rules; they can be also specified separately.

$$U_{el} = \sum_{non-bonded} \frac{q_i q_j}{4\pi\epsilon_0 r_{ij}} \tag{3}$$

is the sum of electrostatic interactions

The remaining terms in (1) are inramolecular interactions due to covalent bonds, covalent angles and torsion angles. They are described below.

For each molecule type used in the simulation a file describing molecular structure and parameters of the force field is needed. The file must have extension *.mmol . Some examples of .mmol files are given in `moldb` directory.

`.mmol` files consist of several parts which are described below. Lines, beginning with "#" are commentaries and they are ignored by the program. Other lines contain parameters in free format. The delimiter between keywords and parameters is one or more spaces. Do not use Tab as a delimiter, it may result in unpredictable behaviour.

2.2 Molecular geometry and non-bonded interaction parameters

The first non-commentary line of a `.mmol` file is the number of atoms in the molecule. After it the corresponding number of lines follows, one line per atom. Each line contains 8 compulsory parameters. They are: 1) atom name in the program; 2),3) and 4) are the initial X,Y,Z coordinates of the atom in the molecular coordinate system, 5) mass in atom units, 6) charge, 7) Lennard-Jones parameter σ (effective atom diameter) in Å, 8) Lennard-Jones parameter ε in kJ/M. Two optional columns may present, the 9-th column with the chemical types of atoms according to the chosen force field and the 10th, with the atom numbers.

This part is concluded by a reference to the molecular structure/ force field which is read by the program and printed in the output. It consists of the first line with a number specifying the number of lines for the reference, followed by the corresponding number of lines with the reference itself.

2.3 Bonds

Two types of bonds are implemented, standard harmonic bond (Note: no factor 1/2 in the potential):

$$U_{harm}(r) = k(r - r_0)^2 \tag{4}$$

and Morse potential:

$$U_{morse}(r) = D\big(1 - \exp(-\rho(r - r_0))\big)^2 \tag{5}$$

Harmonic bonds may be also used for description of the so-called Urey-Bradley term which is a harmonic potential between the 1-3 neighbors (atoms separated by two covalent bonds). This term is included for example into CHARMM force field as a part of interaction potential for covalent angles.

The first line of the bond section of a `.mmol` file is the number of bonds. Then lines, one per each bond, follow with parameters for each bond. Each line consists of the following fields:

- The first is the type of the bond potential. Available values are 0 (harmonic bond), 1 (Morse potential) and 2 (Urey-Bradley term). In the case of the Urey-Bradley term, the interaction is calculated exactly as in the case of a harmonic bond (type 0), but the corresponding bond is not accounted for definition of 1-3 and 1-4 neighbors. It is also not accounted for in constraint dynamics (that is, constraints are not applied for these bonds)
- The second and third parameters are atom numbers which are linked by the bond.
- The fourth field is the equilibrium bond length r_0 in Å. It may be set negative, in such a case the equilibrium bond length will be taken from the corresponding coordinates in the fisrt section of the `.mmol` file.
- The 5-th parameter is the force constant k in $kJ/mol/\mathring{A}^2$
- The 6-th and 7-th parameters are used only for the Morse-type bonds and define the D (kJ/mol) and ρ $(\mathring{A}^{-1})$ parameters of the Morse potential respectively

2.4 Covalent angles

Covalent angles are described by harmonic potential of the angle:

$$U_{ang}(\theta) = k_\theta(\theta - \theta_0)^2 \tag{6}$$

where θ_0 is the equilibrium angle and k_θ is the force constant.

The first line specifies the number of angles.

Next lines give parameters of angles. The fields of each line have the follow meanings:

1),2) and 3) define atom numbers

4) is the equilibrium angle in degrees

5) is the force constant in kJ/mol/rad**2

Note. The CHARMM force field has the so-called Urey-Bradley term which is a harmonic interaction between 1-3 neighbors. This term can be introduced in the list of bonds.

2.5 Dihedral angles

.

In this section, parameters for dihedral angles are introduced for the standard type of torsion potential:

$$U_{tors}(\phi) = K_\phi(1 + \cos(M\phi - \Delta)) \tag{7}$$

Torsional angle ϕ is defined as having value 180° for a trans-conformation.

Other types of dihedral angle potential are supported, they can be introduced in other, optional, sections.

The first line of this section specifies the number of torsion angles of this type. Lines describing torsion potential have the following meaning for the fields:

1)-4) define atoms for the torsion angle

5) is parameter Δ in degrees

6) is the force constant K_ϕ in kJ/mol

7) is the multiplicity factor M

Multiple torsions are supported. Any torsion angle can be repeated any number of times with different parameters, and all they will be summed up in the calculations of forces and energies.

2.6 Optional features of the force field

Description of the optional features of the force field begins with the line defining the type of additional potential features. These are:

- `tors1`

  describes torsion angles in terms of MM3 potential:

  $$U_{tors}(\phi) = K_1(1 + \cos\phi)/2 + K_2(1 - \cos 2\phi)/2 + K_3(1 + \cos 3\phi)/2 \tag{8}$$

  The first line of this section contains the number of torsions of this type followed by lines in which fields 1)-4) define atoms and fields 5)-7) constants K_1, K_2 and K_3 in kJ/M for each torsion angle of this type.

- `tors5`

  describes torsion angles given by the Ryckaert-Bellemans potential:

  $$U_{tors}(\phi) = \sum_{i=1}^{5} K_i \cos^i(\phi - 180) \tag{9}$$

  The first line of this section contains the number of torsions of this type followed by lines in which fields 1)-4) define atoms and fields 5)-9) constants $K_1, ..., K_5$ in kJ/M for each torsion angle of this type.

- `improper`

  improper dihedral potential:

  $$U_{impr}(\phi) = k_\phi(\phi - \phi_0)^2 \tag{10}$$

  field 5) is the equilibrium angle in degrees and 6) is the force constant

- `special`

  introduces a list, over-reading Lennard-Jones parameters for 1-4 interactions for the specified atoms. This list consists of the following:

  the first line - number of atoms in the list;

  other lines: the first field defines atom, the second is σ (Å) and the third parameter is ε in kJ/M.

  Scaling parameter for 1-4 LJ interactions, eventually specified by `slj14` keyword (see below), is applied to these modified LJ parameters also.

- `sel14 <factor>`

  Scale 1-4 electrostatic interactions by this factor

- `slj14 <factor>`

  Scale 1-4 Lennard-Jones interactions by this factor. This applies also to LJ parameters modified by keywords `special` or `par14` of the .mmol file, and to parameters introduced by `Pair_list` keyword of the input file.

- `par14`

  Assign special LJ parameters for 1-4 neighbours

  This parameter follows by a line specifying the number of special 1-4 pairs followed by the corresponding number of lines with four parameters: site numbers of the first and second atom, then LJ parameter σ (Å) and LJ parameter ε (kJ/mol).

  Note: There are several ways to modify LJ parameters for 1-4 interactions. They are applied in the following order (each next definition overrides the previous ones):

  1. Default (Lorentz-Berthelot or those specified in the input file) combination rules are applied
  2. Special Lennard-Jones parameters for each atom (defined by the `special` section of the .mmol file) are applied
  3. Special 1-4 Lennard-jones parameters for atom pairs from the `par14` section of the .mmol file are applied
  4. Global list of LJ interaction parameters introduced by `Pair_list` keyword of the input file is applied

5. All parameters for 1-4 LJ pairs (both modified and unmodified) are scaled by the factor introduced by `slj14` keyword for the corresponding molecular type

- `fSPC`

  This parameter is exclusively for the flexible SPC model. It tells to the program to compute water intramolecular potential according to the paper by Toukan and Rahman, Phys. Rev. B, 31(2) 2643 (1985). It is possible to change internal bond parameters in the .mmol file.

Examples of `.mmol` files are included into `moldb` directory. For big molecules, procedure of writing `.mmol` files may be somewhat automatized by the utility `makemol`.

3 The main input file

3.1 General notes

The main input file consists of keywords followed by parameter(s) Lines beginning with "#" are considered as commentaries. Keywords are case sensitive. Lines which are not recognized as keywords are considered as commentaries, but a warning is given in the output if the first symbol of such line is not "#". The order of keywords is almost unimportant. The only exception is that the information on the number of molecular types (keyword `Mol_types`) should be given before any keyword involving reference to specific molecules and molecular types. For missing keywords, default values are used. Repeated reading of the same keyword overwrites its previous value

3.2 Basic setup

- `Main_filename <name>`

  `<name>` is the base file name for the given simulation. Other files requested or created by the program have this name with various extensions

  This keyword is compulsory.

- `Verbose_level <number>`

  `<number>` - Output control parameter (integer). Suitable values 2-10. The less number, the less you see in the output. Parameters higher 7 used mostly for debug purposes

  default: 5

- `Path_DB <name>`

  `<name>` - Path to the molecular database containing files describing molecular structures and force field parameters (.mmol files)

  default: . (current directory)

- `Visual yes/no`

  Run with visual shell (not included)

  Default: no

3.3 Restart control

- `Read_restart yes/no [ASCII]`

  "yes" - read restart file `<Main_filname>.dmp` and continue old simulation from the restart file. .

  "no" - start a new simulation

  default: yes (this is to avoid non-intentional startup of a new simulation which may rewrite an old restart file possibly containing results of a valuable simulation)

  "ASCII" is an optional parameter. If it is specified, restart file will be read in ASCII format

- `Dump_restart <nsteps> [ASCII]`

  Dump restart file after every `<nsteps>` of the simulation

  "ASCII" is an optional parameter. If it is specified, the restart file will be written in ASCII format.

  Note: Binary format of restart file is preferable since it takes less space and do not lose any precision. ASCII restart file should be used only if you want to continue the simulation on a computer with another architecture

  Default: do not dump restart file

- `Change_T yes/no`

  Change temperature after restart. This option sets up a new value for the target temperature which is specified in the `Nose_thermostat` or `Velocity_scaling` keywords. Otherwise ("no"), the old value of the temperature saved in the restart file is used

  If the target temperature is changed, all the velocities are rescaled by the corresponding factor.

  Default: no

- `Change_V yes/no/slow [num]`

  Change volume after restart. If this option is set to "yes", new values for the box sizes, which are defined by the `Box` or `Density` keyword, will be used.

  In case of ("no"), the values of the box sizes saved in the restart file are used. In case of new run with initial structure from a ".start" file (see `Startup` option), the values of the box sizes specified in the initial structure will be used.

If `Change_V` option is set to "slow", the program starts with the box size specified in the restart file or in the ".start" file with initial structure. Then, during `num` steps, the box sizes are gradually transformed to the values given by `Box` or `Density` keyword. This option is not compatible with constant-pressure simulations.

In all cases when the box sizes are changed, all the distances between the molecules are rescaled by the corresponding factor.

Default: no

- `Check_only yes/no`

  "yes" - Do not run actual simulation but only check all input information. If new simulation, only check input. If keyword `Read_restart yes` is specified, type out results saved in the restart file.

  "no" - Run simulation as specified

  Default: no

- `Zero_CPU yes/no`

  If "yes", set counter of the CPU time to zero

  Default: no

- `Zero_vel yes/no`

  If "yes", set all velocities to zero (at start-up or at restart)

  Default: no

- `Zero_average yes/no`

  If "yes", set all accumulators of averages to zero and clear the history of averages

  Default: no

3.4 System

- `Molecule_types <num_types>`
  `<name> <number> [fixed]`
  `...`
  `<name> <number> [fixed]`

  Which molecules to simulate. `<num_types>` is the number of molecule types. This keyword must be followed by `<num_types>` lines each of which contains at least two parameters: `<name>` and `<number>`

  `<name>` - name of the molecule. File `<name>.mmol` , describing the molecular structure and the force field parameters, must be present in directory specified by parameter `Path_DB`.

  `<number>` - number of molecules of this type.

"fixed" - Option "fixed" signals that these molecules are fixed and will not move in the MD procedure.

This keyword and its parameters (except "fixed") are compulsory

- `PBC <type>`

  Type of periodic boundary conditions

  `<type>` may be:

  `rect` - rectangular (default)

  `hexa` - hexagonal

  `octa` - truncated octahedron

- `Box  <box-x>  <box-y>  <box-z>`

  Specifies simulation box size (in Å). In case of hexagonal boundary conditions, only `box-x` and `box-z` have a meaning. For octahedral boundary conditions only `<box-x>` has a meaning. The keyword cannot be used simultaneously with "Density"

- `Density  <value>`

  Setup initial density. `<value>` is density in g/cm^3. This option cannot be used simultaneously with setting up box sizes. In case of rectangular periodic boundary conditions, a cubic cell is implied. Zero density implies "vacuum simulation" regime with the box size automatically set to 1000 Å

- `Combination_rule  <type>`

  Use another (than Lorentz-Berthelot) combination rule for Lennard-Jones parameters for different types

  `type` may be one of the following:

  `Sigma_geom`

  LJ σ parameter is calculated as a geometrical average of the both types (as for example in the OPLS force field)

  `Kong`

  use Kong rules (J.Kong, J.Chem.Phys., 59, 2464, (1973))

  `Fender`

  use Fender-Halsey rules (B. E. F. Fender and G. D. Halsey, Jr. "*Second Virial Coefficients of Argon, Krypton, and Argon-Krypton Mixtures at Low Temperatures*", J. Chem. Phys. 1881 (1962))

  `Waldman` use Waldman-Hagler rules (M. Waldman and A. T. Hagler "*New combining rules for rare-gas Van der-Waals parameters*", Journal of Computational Chemistry 14 pp. 1077-1084 (1993))

`Mixed` use mixed combination rules from paper: A. Nikitin, Y. Milchevskiy, A. P. Lyubartsev "*AMBER-ii: New Combining Rules and Force Field for Perfluoroalkanes*" J. Phys. Chem. B, 119, 14563-14573 (2015)

Default: use Lorentz-Berthelot combination rules

- `Pair_list [FF_file] <file_name>`

  Introduce special Lennard-Jones parameters for pairs of atoms given in file `<filename>`.

  Without `FF_file` option the list of pairs is directly composed from the site numbers given in file `<filename>`. Each non-commentary line of this file consists of two integer numbers specifying the global site numbers of two atoms, followed by two real numbers having the sence of LJ parameters σ and ε for the given pair of sites, and eventually followed by two other real numbers which are LJ parameters σ and ε for 1-4 bonded interactions. This list overwrites the Lennard-Jones parameters defined by the combination rules and eventually modified by parameters in the `.mmol` files.

  If option `FF_file` is given, the list of pairs is composed from the `LJ_PAIRS` section of the force field file given by parameter `<filename>` (same force field file as used by the `makemol` utility). The pairs of atoms with force field atom types (given by the 9-th colomn of the `.mmol` files) which match atom types in the `LJ_PAIRS` section of the force field file are included into the list of pairs for special Lennard-Jones interactions.

- `El_field <ampl> <freq>`

  Put the system in a homogeneous time-dependent electrostatic field

  $$E(t) = A\cos(2\pi\omega t) \tag{11}$$

  where `<ampl>`$= A$ is given in V/cm and `<freq>` $= \omega$ in Hz.

- `G-force <type> <g-force>`

  A gravitation-like force (acting along Z-axis on each atom proportionally to its mass) is added for molecules of type `<type>`. Parameter `<g-force` gives the value of acceleration g in $10^{12}m/s^2$. The force can be applyed for several molecular types, is this case the keyword should be repeated for each type.

3.5 Ensemble

- `Nose_thermostat <temp> <relax.time>`

  Specify whether to use the Nose thermostat. The option is incompatible with "velocity scaling". Two parameters are compulsory:

  `<temp>` - target temperature

`<relax.time>` - relaxation time in femtoseconds

Note: if simulation is restarted from a restart file, the old value of temperature specified in the restart file is normally used, ignoring value of this keyword. To override this, use keyword `Change_T`.

- `Velocity_scaling  <temp>  <dt>`

  Specify whether the temperature is regulated by the velocity scaling. The option is incompatible with "Nose thermostat". Two parameters are compulsory:

  `<temp>` - target temperature

  `<dt>` - admissible deviation. If the temperature deviates from the target by more that "dt", velocities are rescaled to fit the target temperature.

  Note: if simulation is restarted from a restart file, the old value of temperature specified in the restart file is normally used, ignoring value of this keyword. To override this, use keyword `Change_T` .

- `Separate_thermostating  yes/no`

  Specify whether to apply temperature control to the whole system (no) or separately to each molecular type (yes). The option works both for Nose thermostate and for temperature control by velocity scaling

  default: no

  Note. At this option, the total momentum is not conserved. Use with care. See also `COM_check` keyword.

- `COM_check  yes/no  [num]`

  Specify whether to remove motion of the total center of mass. If parameter "num" is zero or negative, motion of the center of mass is removed only at the program start. If this parameter is positive, it gives the number of MD-steps after which the removal of center of mass motion is repeated

  Default: yes 0

- `Barostate_NH <pres> <relax_time>`

  Specify whether to use the Nose-Hoover barostate. Option works only if Nose thermostat is specified. Without the keyword, constant volume simulations are implied. Two parameters are compulsory:

  `<pres>` - pressure in bar (atm)

  `<relax_time>` - relaxation time in femtoseconds

- `Barostate_anisotropic  yes/no/XY [<Px> <Py> <Pz>]`

  Specify whether to apply pressure control isotropically (no), separately to each direction (yes), or separately in XY-plane and along Z-axis (XY). The option has effect only if constant-pressure simulations (barostate) is specified

`[<Px> <Py> <Pz>]` is pressure in each direction. If these three values are specified, they overwrite the value given in `Barostate_NH` command. If they are not specified, pressure given in `[<Px> <Py> <Pz>]` is used in each direction.

default: no

note Anisotropic pressure control has a sense only if your system is really anisotropic by physical reasons

3.6 Molecular Dynamics

- `Time_step <value>`

  Time step in femtoseconds. This is the long time step in case of the double time step algorithm.

  Default: 2 fs.

- `Number_steps <number>`

  How many MD steps to run.

- `Double_timestep <number>`

  Double time step algorithm by Tuckerman et al. `<number>` is the number of short time steps in one long. The keyword cannot be used simultaneously with `Constrain`.

- `Constrain <toler> <i_1> <i_2> <i_3> .... <i_ntypes>`

  Use SHAKE algorithm to constrain the bond lengths. The option cannot be used with `Double_timestep`. `<toler>` is the tolerance level.

  `<i_1> <i_2> ... <i_ntypes>` are numbers 0 or 1, which specify whether (1) or not (0) to apply constraints to molecules of each species.

- `R_cutoff <value>`

  Cut-off (in Å) for the Lennard-Jones and real-space part of the electrostatic forces.

  Default: 12 Å

- `R_short <value>`

  Cut-off (in Å) for Lennard-Jones forces computed each short-time step (has an effect only in double-time step algorithm)

  Default: 5 Å

- `Neighbour_list <number>`

  Update the list of neighbors (Verlet list) every `<number>` steps

  Default: 10

- `Electrostatics <type>  [<A> <B>]`

  How to treat electrostatics

  `<type>` may be Ewald (default), RF (reaction field) and Cutoff

  For Ewald: A is α/R_{cutoff}, where α is the Ewald convergence parameter. Precision of the real-space Ewald part is determined by $erfc(A)$. B defines the number of terms in the reciprocal part. It cuts the reciprocal series when expression in the *exp* of the reciprocal part exceeds B. Recommended values A = 2.5 – 3; B = 7 – 10.

  For reaction field, A is the dielectric permittivity and B is the Debye screening length in Å. Setting the Debye length to 0 means an infinite Debye length, i.e non-conducting solution.

  If `<type>` is "Cutoff", parameters A and B are not necessary, and no special treatment of electrostatic forces out of R_{cutoff} takes place.

  Default: Ewald method with A=2.8 and B=9.

- `Cut_forces   <value>`

  If this option is specified and the absolute force acting on any atom exceeds the specified `<value>`, the force will be cut to this level while maintaining the direction. The value is given in Å and has a sense of the maximum allowed additional displacement during one time step (F*dt**2/m) caused by this force

  Default: Do not cut forces

- `Bind_atoms   <file> <deviation>`

  If this keyword is specified, atoms defined in file `<file>` will be bound to corresponding positions (given in the same file) by a harmonic potential with characteristic deviation given by `<deviation>`

- `Control_temp  <factor>`

  Activate control of temperature during the simulation. If instant temperature deviates from the target (thermostat) temperature by factor more than `<factor>`, the program stops simulation and prints current configuration of the system

  Default: Off

3.7 Startup

- `Startup <type> [<optional parameters>]`

  How to set up initial coordinates. This option has an effect only if `Read_restart` was specified as "no"

  `<type>` may be one of the following:

  `xmol` (default) - the input configuration is taken from file `<Main_filename>.start` (see keyword "`Main_filname`") which should be in "XMOL" format The

second (commentary) line of this file may contain box sizes (which follow after keyword `BOX:`). They will be used as actual box sizes if keyword "`Change_V`" is not set or set to "no". If keyword "`Change_V`" is set to "yes" or "slowly" , the box sizes specified by keywords "`Box`" or "`Density`" will be used.

`xyz` - the input configuration is taken from coordinates written as three columns of X Y Z coordinates in `<Main_filename>.start` file

`Mol_COM` - molecular center-of mass coordinates are taken from

`<Main_filename>.start` file, as three columns of X Y Z coordinates.

`FCC` - molecular center of mass coordinates are put on a FCC lattice

`Cubic` - molecular center of mass coordinates are put on a cubic lattice

`Cyl_hole <radius>`

`Sph_hole <radius>`

Molecules, except those defined in option `Start_rot` are distributed outside sphere or cylinder of radius `<radius>`

In all cases, except "xmol" and "xyz", atom coordinates are build around molecular center-of-mass using local coordinates specified in the corresponding .mmol files.

- `Start_rot  <i_1> <i_2> <i_3> ... <i_ntypes>`

  Tells how the molecular coordinates defined in .mmol files are treated at startup:

  If Startup options "xmol" or "xyz" are specified, keyword `Start_rot` do not have any effect.

  Startup option "FCC" or "cubic":

  If `i_n`=0, molecules with coordinates from -mmol file are rotated randomly

  If `i_n`=1, coordinates of the molecule of this type are taken from .mmol file without any change.

  If `i_n`=2, coordinates of the molecule of this type are taken from .mmol and translated without rotation so that the molecular center of mass will be at zero.

  Startup options "`Cyl_hole`" and "`Sph_hole`":

  Molecules which should be placed inside cylindrical or spherical hole, are marked by parameter `i_n` = 1 or `i_n` = 2. Initial coordinates of atoms of such molecules are taken from the corresponding .mmol file and it can be only a single molecule of this type. In case `i_n` = 2 the molecular coordinates are translated so that center of mass is set to the coordinates origin. Other molecules (with `i_n = 0`) are distributed outside the spherical or cylindrical hole.

  Cases `i_n` > 0 have a sense only if only one molecule of this type is present.

  Default: all `i_n`=0 (rotate molecules randomly)

- `Gather`

  Gather each molecule in one place (if its atoms were spread over different periodical images). Works both at start-up and at restart.

  Default: use coordinates as they are.

3.8 Expanded Ensemble mode

Expanded ensemble (EE) mode can be used for calculations of solvation free energies by gradual removal or insertion of molecules in the system. The theory and computational details behind the expanded ensemble method are described in papers (J.Chem.Phys.96, 1776 (1992); Molec.Simulations , 18, 43 (1996)). Details of the current implementation are described in Jämbeck et all, J.Comput.Chem.,34,187 (2013).

The expanded ensemble methodology implemented in MDynaMix from v.5.2.3, includes the Wang-Landau algorithm, Wang F., Landau D. P., Phys. Rev. Lett. **86**, P. 2050–2053 (2001)), for optimization of the balancing factors. Also, from v.5.2.5, a different scheme of interaction change of the choosen solute molecule with the rest of the system, based of soft core interaction potentials (see e.g. L.Luder, R.Kjellander, J.Phys.Chem.B., 110, 15514 (2006)) , is implemented. Particularly, the interaction potentials between all pairs of the solute and solvent atoms are changed with the insertion parameter α, $0 \leq \alpha \leq 1$ according to:

$$V^{Ss}(\alpha, r) = \alpha^{w1}\left[4\varepsilon\left(\left(\frac{\sigma}{r + b(1-\alpha^{w2})}\right)^{12} - \left(\frac{\sigma}{r + b(1-\alpha^{w2})}\right)^{6}\right) + \right.$$
$$\left. + \frac{q_i q_j}{4\pi\epsilon_0(r + b(1-\alpha^{w2}))}\right] \quad (12)$$

The coupling parameter α assumes a number of fixed values between 0 and 1, where "1" corresponds to the solute molecule properly inserted in the solvent, while value "0" correspons to the fully eliminated solute (which then can be considered as an a gas phase). The program computes probability distribution over subensembles with different values of α, from which the free energy difference can be obtained. Additional bias for transition probabilities between subensembles is given by the balancing factors (biased potential over the insertion parameter), which need to be tuned in order to provide reasonable homogenious distribution of probabilities over subensembles. The Wang-Landau procedure is by default used for tuning of the balancing factors, but there is also possibility for manual tuning.

Note: Special care should be taken in MDEE simulations of molecules having zero Lennard-Jones potential for hydrogen atoms (like SPC or TIP3P water). Simulation may become unstable when repulsive potential between oxygens becoms weeker while electrostatic attraction between hydrogen and oxygen still present. It is advisable to prescribe some small Lennard-Jones potential to such hydrogens.

The expanded ensemble is specified by the following keywords:

- `MDEE <num_ensembles> <mode>`

  Run expanded ensemble simulations with `<num_ensembles>` subensembles, Parameter `<mode>` can be one of:

  `manual` : manual setup of EE parameters, that is α-points and balancing factors which remain fixed during the simulation

  `WL` : balancing factors are optimized during the simulation according to Wang-Landau algorithm (J.Comp.Chem., 34, 187 (2013))

  `L2MC` : Implement L2MC algorithm for transitions between subensembles, in addtion to WL algorithm (J.Comp.Chem., 42, 787 (2021)

  `auto`: α poins are set with equal separation between them within the [0:1] interval, balancing factors are tuned automatically by the Wang-Landau algorithm and L2MC algorothm for transitions between subensembles is used

  In the case parameter `mode` is not set to `auto`, the `MDEE` command should be followed by exactly `num_ensembles` lines, with two number on each: the value of coupling parameter α and balancing factor, corresponding to this subensemble. Values of balancing factors are given per inserted molecule in units of "kT".

  In the case of automatic setup of the expanded ensemble parameters, the set of coupling parameters α is choosen uniformly in the range [0:1] (that is, $\alpha_m = (M-m)/(M-1)$ where M is the total number of subensembles `<num_ensembles>`.

  Default mode: `auto`

- `EE_typ <i_typ>`

  Specifies that the molecules of type `i_typ` are those which are inserted/deleted in the EE procedure. **This keyword is required!**

  In the case of automatic optimization of the balancing factors (mode `auto`, `WL`, `L2MC`), additional keywords can be used:

- `Num_pass <value>`

  Defines how many times the system should walk between the extreme subemsembles and back in order to complete one WL iteration. Default is 2.

- `Num_WL <value>`

  Defines the number of WL iterations. After this number of WL iterations is carried out, the optimization of balancing factors is completed, and the program proceeds with EE procedure with fixed balancing factors. Default is 10 iteration.

- `WL_parameters <increment> <factor>`

  Sets up initial increment in the Wang-Landau optimization of balancing factors, and scaling factor by which the increment is scaled after each WL iteration.

  Default values: `increment = 0.1`, `factor = 0.5`.

  In case of automatic WL tuning of the balancing factors (mode `auto`) the program starts expanded ensemble simulation with zero balancing factors. In case of mode `WL` the initial values of the balancing factors are introduced from the input. After each attempt of visiting a subensemble, an increment is added to the corresponding balancing factor. After walking between extreme subensembles at least `Num_pass` times, a WL series (sweep) finishes and the value of the increment is decreased by a factor (default value 0.5, see keyword `WL_parameters`).

  Notes:

  1. If WL procedure has not performed necessary number of iterations (keyword `Num_WL`), the program gives warning in the output "Expanded ensemble simulation not converged yet". Then the computed free energy values may not be accurate or not relevant at all. It is always recommended to continue simulation long enough after the balancing factors are optimized.

  2. While evaluating results, ensure that the system has walked at least several times between the extreme subensembles (the first and the last). Look at the "table of transition" in the end of the output.

  3. Pay attention at the acceptance ratios for transitions between subensembles, they can be found in the last two columns of the section "Distribution over subensembles" in the output. The acceptance ratios should not be too low. If they are too low (a few percent or less), the number of subensembles should be increased.

- `EE_start-in <start_in>`

  Start EE simulation in subensemble number `<start_in>`. Default: in the last subensemble (that is, with completely deleted EE molecule(s))

- `EE_freq <steps>` Try EE transition after `steps` MD steps.

  Default: 10

- `EE_new`

  Change balancing factors after restart (manual mode only). Otherwise the old values written in the restart file are used.

- `EE_offset <b-value>`

  Value of parameter b in equation (12)

  Default: $b = 3\mathring{A}$.

- `EE_power <w1> <w2>`

  Values of parameters $w1$ and $w2$ in equation (12)

  Default: $w1 = 1$; $w2 = 1/3$.

- `Probability_threshold <P_low>` Assing value `P_low` to computed subensembles probabilities which are lower than this number

  Default: 0.001

3.9 Path Integral mode

Path integral mode can be run only in parallel using number of processors equal to the number of beads in the path integral representation of quantum particles. The number of beads should be even.

- `PIMD <num. of beads>`

  Use PIMD mode with the given number of beads

- `PI_thermo <temp> <tau> <chain>`

  Use Nose chain of thermostats for each atom. This is preferable option instead of the common Nose thermostat for all atoms. `<temp>` is the target temperature, `<tau>` is the relaxation time for the thermostats and `<chain>` is the number of thermostats in each chain. If this option is specified, Nose_thermostat option is ignored.

3.10 Properties

- `Output <number>`

  At output level >= 5, type line(s) with some data (time step, energies, etc) each `<number>` of steps

  Default: 1

- `Average_freq <number>`

  Defines how often to collect averages: various internal degrees of freedom, partial energies, probabilities of subensembles in the expanded ensemble mode.

  Default: 1

- `Serie_average <number>`

  Compute and remember intermediate averages over series of `<number>` steps

  Default: 10000

- `Average_from <number>`

  Begin final averaging over series (see `Serie_average` keyword) from series with the given `<number>`

- `Average_internal  yes/no`

  Whether to compute average values of all bond lengths, angles and their energies

  Default: no

- `Dump_XMOL  yes/no`

  Dump the final configuration in "XMOL" format

  Default: no

- `Trajectory  <format> <step> <num_conf> <list>`

  Dump trajectory.

  `<format>` may be one of:

  `bincrd` - binary with coordinates only

  `binvel` - binary with coordinates and velocities (in Å/fs)

  `asccrd` - ACSII (XMOL-format) with coordinates only

  `ascvel` - ACSII (XMOL-type format) with coordinates and velocities

  `ascfor` - ACSII (XMOL-type format) with coordinates and forces (in kJ/mol/Å)

  `ascfvl` - ACSII (XMOL-format) with coordinates, velocities and forces

  `<step>` is time interval (in femtoseconds) between the configurations saved in the trajectory file

  `<num_conf>` - number of configurations in each trajectory file. After filling `<num_conf>` configuration in a trajectory file, the program begins to write configurations in the next file. Trajectory files acquire extensions .001, .002, .003, etc

  `<list>` - list (in the form of numbers 0 or 1 for each molecule type) which signals whether to include molecules of this type into trajectory. `<list>` may be set also to "all" (all molecules)

  Units for velocities in the trajectory file are Å/fs.

- `Bond_list yes/no`

  Generate list of all bonds and put it in file bond.list

  Default:no

- `End`

  end of the input file. This line is compulsory

4 Compilation

The program can be installed in any user-defined directory. After copying the distribution file into chosen directory, unpack it:

`tar xfvz md530.tar.gz`

A new directory `mdynamix` appears. Change to this directory.

Before compiling, you may need to change sizes of working arrays defined in the file `dimpar.h`. The values of parameters depends both of molecular system you want to simulate and on the computer in hands.

The are several Makefiles for different architectures. Choose the most suitable and edit it if necessary. Usually options for optimization can be tuned to improve performance. Then run "make" which invoke one of these Makefiles. The Makefiles differ mostly by compiler used and subroutine to count cpu time.

4.1 Sequential execution

These files produce executable "md"

`make default`

calls `Makefile.f77` . This should work for any standard f77 compiler called `f77`, with exception of accounting cpu time which is substituted by a "dumb" subroutine (substitute your own counter of cpu time in file `cpu_dummy.f`)

`make linux`

or

`make gfortran`

calls `Makefile.gfortran`. This can be used on Linux or other systems with gfortran fortran

`make intel`

calls `Makefile.ifort`. This can be used on Linux with free (at least now) Intel compiler `ifort`. Performance options, such as `-xP` for processors with sse3-instructions, may substantially speed up the program.

`make pgi`

calls `Makefile.pgi` Calls Portland Group Fortran compiler `pgi`.

`make static`

calls `Makefile.static` Creates static executable using gfortran compiler

4.2 Parallel execution

Note that these makefiles are strongly dependent on how the MPI library is installed. The name of executable is `mdp`

`make mpi`

Calls `Makefile.mpi` . This can be used at "standard" MPI installation which uses mpif77 script to call the correct Fortran compiler and link MPI-libraries.

4.3 Other makefiles

5 Execution

The program compiled for one processor can be started from a command line:

`md md.in > md.out`

where `md.in` is the main input file and `md.out` is output. Names of input and output files may be arbitrary

For parallel execution, use a command like

`mpirun -np 8 mdp md.in > md.out`

where number of processes is 8 and `md.out` is name of the output file. Other options for `mpirun` command can be given. Often `mpirun` can be invoked only from a queue batch script, see the rules in your computer center. Sometimes command `poe` is used to start a parallel program

If the argument with the name of the input file is absent, the program looks for file `md.input` and use it as input.

5.1 Files used by the program

Table 1: Files used by the program. "<fname>" is the same for all files in the simulation

file	I/O	required	description
Main input	I	yes	the main input file
MD log file	O	yes	the main output file
`*.mmol`	I	yes	files defining molecular structures and the force field
`<fname>.start`	I	opt	start configuration
`*.dat, *.ff,`	I	opt	files for optional features (LJ pairs etc)
`<fname>.dmp`	IO	opt	restart file
`<fname>.xmol`	O	opt	final configuration
`<fname>.00n`	O	opt	trajectory files

5.2 Memory considerations

Parameters defining arrays boundaries are given in `dimpar.h` file. They define also the required memory. They can be decreased to save memory or increased if they are not enough for the simulated system.

One of the most memory-consuming arrays is the list of the atom pairs within the cutoff distance. Its size is defined by parameters NTOT (maximum number of atoms) and NBLMX - maximum number of neighbors (within cutoff) for each atom. Since the list of neighbors is distributed among the available nodes, this parameter can be decreased in the case of parallel execution.

Much memory can be also taken by the array with intermediate averages for different parameters. Its size is NRQS*LHIST, where NRQS is maximum

number of calculated different averages and LHIST is the maximum number of series for which these averages are calculated. It may be really big for large macromolecules, if keyword `Average_internal` is set to `yes`. In this case, all bond lengths, covalent and torsion angles are calculated and remembered.

The program checks correspondence of array boundaries to the input data and stops if something is wrong. After correction of `dimpar.h`, the code must be recompiled.

5.3 Program structure

The program consists of several Fortran files, each contains one or several Fortran units. The files are:

- `main.f` - the main module.

  This is the main program unit taking care of all what follows.

- `dimpar.h` is the file defining maximum sizes of working arrays. The code must be recompiled after any change in this file.

- `prcm.h` is a header file common for most of program units. It contains static dimensioning of the working fields and definition of all global parameters. Practically all modules depend somehow on it.

- `input.f` This file contains subroutines which read the main input file and .mmol files.

- `setup.f` This file setup units, reference arrays and data structures

- `mdstep.f` This file contains MD integration algorithms, including thermostats and barostats as well as SHAKE algorithm for constraint dynamics

- `forces.f` This file contains subroutines responsible for calculations of different forces. Forces are divided into two groups: slow and fast forces. This division is essential only if the double time step algorithm is used. Procedure for updating the lists of neighbors is also included into this file

- `mpi.f` contains collection of MPI calls for creating parallel executable code. This is the standard version of this file, which uses real*4 size of data during communications.

  `mpi_cray.f` is Cray version of MPI calls for parallel execution

  `mpi_double.f` is a version of MPI calls for parallel execution with double precision for the data transfer

  `mpi_safe.f` is a version of MPI calls with some additional controls for communications.

  `scalar.f` is collection of dummy MPI calls which is used in a single-processor version of the code

- `restart.f`

  This file contains subroutines responsible for writing and reading restart file, dumping trajectories and other operations with input/output files
- `aver.f` collecting averages, including RDFs
- `pimd.f` Path Integral molecular dynamics
- `tcf.f` calculation of time correlation functions (obsolete)
- `service.f` is a collection of some procedures specific for molecular dynamics
- `util.f` is a collection of some other auxiliary procedures
- `cpu_*.f` are different variants of counting cpu time; `getcpu.c` is a C-function to count CPU-time

6 *Makemol* utility

6.1 General organization

The `makemol` utility is used to generate a `.mmol` file (containing information on both molecular structure and the force field) from two files: a force field (`.ff`) file, with information about force field parameters, and a molecular structure file (`.smol` or `.mol2`), which contains only structural information about the molecule (initial atom coordinates, the partial atomic charges, the force field atom types (which should match the types in the force field file) and the list of covalent bonds. The utility constructs the list of covalent angles and torsion angles, and pick up the force field parameters from the force field file. If some parameters are not found in the force field file, the corresponding line in the resulting `.mmol` file is marked with an exclamation mark, and a note is written on the screen and in the `makefile.log` file.

The list of Improper torsions is not generated automatically. If impropers exist for the given molecule, they should be added into `.mmol` file manually.

The molecular structure file can be in one of four formats and have the corresponding file name extension:

- `.smol` : MDynamix simple molecular format, explained below
- `.mol2` : TRIPOS `.mol2` format, compatible with output of *Avogadro* molecular editor and *Antechamber* utility of *AmberTools*.
- `.alc` : Alchemy-like format (depreciated)
- `.crd` : Old Charmm .crd format (depreciated)

Usage:

```
makemol <mol structure file>  <force field file>
```

The format of molecular structure file is determined by the file name extension.

6.2 The force field (.ff) file

All the lines beginning with "#" are considered as commentaries.

The file consists of a number of sections; each beginning with a keyword. Each new keyword means the end of the old section and beginning of the new section corresponding to this keyword. The order of section must be kept. All energies are given in kJ/mol, distances in $\mathring{A}$, and angles in degrees.

- `begin_definitions`

  `< an orbitrary number of lines >`

  `end_definitions`

  This optional section is optional. It contains list of atom types with their masses, and a number of directives.

  Directives:

  `FACTORS <fudge_LJ> <fudge_Q>`

  This defines scaling factors for 1-4 Lennard-Jones and electrostatic interactions

  `No_UB` : do not include Urey-Bradley term even if they are present

  `SKIP_ABS` : skip writing angles and torsions into `.mmol` file parameters for which are not found.

  Other lines are in the form:

  ¡atom type¿ ¡mass¿ [commentary]

  These lines determine masses for atom types, which will be written in the `mmol` file. If masses of atoms are not determined in this section, the values of masses will be guessed from the first letters of the atom type names.

- `BONDS`

  Contains parameters for covalent bonds. Each line consists of four or six parameters. The first two parameters are the force field atom types. The third parameter is the force constant k of the harmonic potential (4) and the fourth parameter is the equilibrium distance r_0. The presence of 5-th and 6-th parameters means that the corresponding bond should be described by a Morse potential, they have a sense of D and ρ parameters of the Morse potential (5) correspondingly.

- `ANGLES`

  Contains parameters for covalent bonds. Each line consists of 5 or 7 parameters. The first three parameters are the force field atom types. The fourth parameter is the equilibrium angle, the 5th parameter is the force constant k. The 6-th and 7-th parameters are optional, if they are present, they define the Urey-Bradley (UB) potential which is in fact a distance-dependent harmonic potential between 1st and 3rd atoms. The equilibrium distance for UB potential is given as a 6-th parameter and

the force constant as a 7-th. `makemol` puts the UB term into the "bond" section of the `.mmol` file.

- `TORSIONS`

  Contains parameters for torsion potentials of "standard" type (7). Each line consists of 7 parameters, the first four define the force field atom types, and parameters from 5-th to 7-th are Δ, K_ϕ and M correspondingly.

  This section allows definition of multiple torsions and wildcards (denotes as "X"). The wildcards can be used only for the 1st and 4-th atoms in the list. The following rules are applied. If an explicit torsion (without wildcards) is specified, lines with wildcards matching this torsion are ignored. If lines with explicit torsions are repeated for the same set of atom types, they define multiple torsions, that is the total torsion potential is a sum of several terms of type (7).

- `NONBONDED`

  Contains Lennard-Jones parameters for non-bonded interactions. Each line consists of 3 or 5 parameters. The first parameter is the force field atom type. The second and third parameters are σ and ε of the Lennard-Jones potential. If the 4-th and 5-th parameters are present, they define σ and ε parameters for 1-4 Lennard-Jones interactions.

- `PAR14`

  Defines list of special LJ parameters for atom pairs involved in 1-4 interactions. Each line contains 4 parameters, the forst two defines two atom types, and the next two defines Sigma and Epsilon for LJ interactions between atoms of this pair when they atoms are 1-4 neighbours.

- `LJ_PAIRS`

  Defines Lennard-Jones parameters for cross-interactions which overwrite combination rules. Each line contains 4 parameters: two first defines atom types, the 3rd defines LJ sigma parameter and the 4-th define LJ Epsilon parameter. Content of this section is not used in generation of `.mmol` files, but it is used during molecular dynamics run if parameter `Piar_list FF_file <value>` is present in the md input file.

- `END`

  The last line of the file

6.3 char2mdx

This utility (being compiled from file char2mdx.f) converts CHARMM force field parameter file (.inp or .prm) to MDynaMix force field file (.ff). Usage:

```
char2mdx < par_all36_lipid.prm > Charmm36_lipids.ff
```

6.4 The molecular structure (.smol) file

This file contains information on molecular structure, partial atomic charges and the force field atom types. All the lines beginning with "#" are considered as commentaries. The first non-commentary line defines the number of atoms in the molecule. Then follows the corresponding number of lines describing each atom. Each line consists of 6 parameters. The first one is the name of the atom. Parameters 2 - 4 are the initial X, Y and Z coordinates of atoms. The 5-th parameter is the partial atom charge, and the 6th parameter is the force field atom type. `Makemol` just rewrites parameters 1-5 into corresponding sections of the `.mmol` file, and uses the force field atom type (6th parameter) in order to find the force field parameters from the `.ff` file.

`Makemol` defines masses from the `begin_definitions` section of the force field file, and if it is not present, from the one or two first letters in the atom name (the first column of `.smol` file). If the program cannot recognize the element, it complains about that; in such a case the mass of the atom must be put manually into resulting `.mmol` file.

After the description of atoms, a section describing the bond structure follows. The first line of this section is the number of bonds. Then the corresponding number of lines follows, with two numbers in each line defining numbers of atoms bound by the corresponding bond. From this list, `Makemol` builds lists of covalent angles and torsions angles, and substitute the corresponding force field parameters.

7 *Tranal* utility

7.1 General organization

TRANAL package consists of a number of utilities for analyzing of atom trajectories generated by molecular dynamics programs. The package was initially constructed to work primarily with MDynaMix molecular dynamics program, but from later versions can accept trajectories written in some other formats: XMOL (.xyz) format, .trr, .xtc, PDB trajectories from GROMACS and NAMD binary trajectories (.dcd).

It is supposed that in all cases, the atoms and molecules are arranged in the following way (the same as in MDynaMix program):

`<molecules of type 1><molecules of type 2>...`

in each molecule type:

`<molecule 1><molecules 2>...`

in each molecule:

`<atom1><atom2>...` (the same atom order must be in all molecules of this type)

Additionally, a term "site" is determined. If one chooses one molecule of each type and set them one after another (in the same order as in the trajectory), then the number of each atom in this sequence would correspond to the "site" number.

Each of utilities in the TRANAL suite consists of two parts: the main block `tranal_base.f`, which reads trajectories, and a specific part which makes required analysis. Correspondingly, the input file for each utility consists of two parts: the first one which determines how to read trajectories, and the second part containing parameters for the required analysis.

Currently, the following analyzing utilities are included:

- **bileldens** - for membranes or bilayers: computation of mass density, electron density, and electrostatic potential across membrane
- **diel** - calculation of average dielectric permittivity
- **diffus** - calculation of the mean square displacement and the self-diffusion coefficients
- **dryrun** - just reading trajectories and optionally rewriting them to XMOL format
- **h-bonds** - identification of hydrogen bonds and their statistics
- **latdiff** - computation of 2D mean square displacement and lateral diffusion for membranes or bilayers
- **order** - computation of the order parameter and angular distribution of specific molecular vectors
- **rdf** - computation of radial distribution functions
- **rg** - computation of gyration radius and distance between two given atoms in a molecule, as well as their distributions
- **sdf** - computation of spatial distribution functions
- **torsion** - computation of distributions of torsion angles
- **torsion2** - Ramachandran plot for two torsion angles
- **restime** - Calculation of residense time
- **trtcf** - computation of some time correlation functions

All tranal utilities can be compiled by setup.sh script in the tranal directory. The compiled utilities can be found in tranal/bin folder.

To read .xtc and .trr trajectories in Gromacs format, tranal uses `xdrfile` library

Eventually, sizes of arrays may be needed to adjust, by editing **tranal.h** file, as well as sizes of arrays in each specific procedure.

Contributions for analysis of other properties are welcomed.

7.2 tranal_base: reading trajectories

This part is common for all types of trajectory analysis. It reads the first part of the input file and then reads trajectories accordingly. The first part of the input file is written in "NAMELIST" format which looks like:

```
$TRAJ
parameter=value(s),
...
$END
```

"TRAJ" is the name of this NAMELIST section.
The following parameters must be defined:

- NFORM = <format>

  where <format> is one of:

  - MDYN - MDynaMix binary trajectory (default)
  - XMOL - XMOL trajectory. It is implied, that the commentary (second) line of each configuration is written in the format:

    (char) <time> (char-s) BOX: <box_x> <box_y> <box_z>

    where (char) is any character word, <time> is time in fs,
    <box_x> <box_y> <box_z> (following after keyword BOX) are the box sizes.
  - PDBT - PDB trajectory as generated by "trajconv" utility of GROMACS simulation package
  - DCDT - DCD trajectories generated by NAMD package
  - —XTC— - .xtc trajectory from Gromacs
  - —TRR— - .trr trajectory from Gromacs

- FNAME = <file_name>

  In case of multiple trajectory files, set the base name of the trajectory files. The trajectory must be then written as a sequence of files <file_name>.001 , <file_name>.002 and so on, the largest possible number being <file_name>.999 .

  In case of a single trajectory file (indicated by parameter NFBEG = -1) set the full file name (with extension)

- PATHDB = <value>

  Directory with molecular description files (.mmol). Default is the current directory (.) .

- NTYPES = <value>

  Number of molecule types in the trajectory

- `NAMOL = <name1> [,<name2>,...]`

  `NTYPES` names of molecules. It is supposed that files `<name1>.mmol`, `<name2>.mmol`,... , or alternatively `<name1>.itp`, `<name2>.itp`,... describing the molecules are present in the directory defined by `PATHDB`. Format of .mmol files is the same as for MDynaMix. Format of .itp files is the same as in Gromacs, with exception that no "include" statements is allowed. Tranal looks first for .mmol files, than for .itp files. In case of absence of any of them, the program still can proceed with the following reservations: i) correct parameters `NSPEC` and `NSITS` are given for all molecules in the trajectory ii) the masses of all atoms are set to 1, the charges to 0, and atom nanes are undetermined.

- `NSPEC = <n1>[,<n2>,...]`

  Number of molecules of each type (`NTYPES` numbers). This parameter is not necessary in MDynaMix binary trajectories.

- `NSITS = <n1>[,<n2>,...]`

  Number of atoms on each molecule of each type (`NTYPES` numbers). This parameter is not necessary if .mmol or .itp files for each molecular type are provided.

- `NFBEG = <value>`

  Number of the first trajectory file (integer between 0 and 999). If this number is negative, a single trajectory file with name given by parameter `FNAME` is used.

- `NFEND = <value>`

  Number of the last trajectory file (integer between 0 and 999)

- `IPRINT = <value>`

  Defines how much you see in the intermediate output. The final output with analysis of results does not depend on it. Default value is 5.

- `BOXL = <x-box-size>`
  `BOYL = <y-box-size>`
  `BOZL = <z-box-size>`

  define the box size if it is not present in the trajectory (implies constant-volume simulation)

- `BREAKM = <value>`

  Defines which break in trajectory (in ps) is allowed to consider the trajectory as continuous. The parameter is used in analysis of time-dependent properties.

- `LVEL = <logical>`

  Accept a logical value. `.true.` signals that the trajectory may contain velocities, which will then be read for the analysis.

- `LXMOL = <logical>`

  Accept a logical value. If .true., rewrite trajectory in XMOL format

  `FXMOL = <name>`

  File name for output XMOL trajectory

- `ISTEP = <value>`

  Specifies that only each `ISTEP`-th configuration from the trajectory is taken for the analysis

- `LBOND = <logical>`

  Accept logical value. If .true., information about bonds is read from .mmol files

- `VFAC = <value>`

  Multiply velocities from the trajectory by the given `<value>`. This can be used for trajectories prepared by other MD programs than MDynaMix, in order to bring velocities to Å/fs (the units which are used in the analysis). In MDynaMix trajectories, velocities are already in Å/fs. Default is 1.

7.3 Computation of the electron density and electrostatic potential in bilayers: bileldens.f

This computation is made by `bileldens` utility. It is supposed that a membrane-like system is oriented in XY-plane. The utility computes the electron density, the mass density, the charge density, and the electrostatic potential across membrane in Z-direction.

Input parameters for this utility follow after the trajectory parameters in the NAMELIST block `ELDEN`:

```
$ELDEN
parameter=value(s),
...
$END
```

The following parameters are used:

- `FDENS = <filename>`

  Defines name of the output file

- `IRT = <int.num>`

  Defines the type number of "lipid" molecules. The middle plane of the bilayer is calculated from the center-of-mass Z-coordinate of molecules of this type. Eventually the molecules are transfered across the periodic boundaries to keep the whole bilayer within the same periodic cell.

- `NA = <int.num>`

  Defines the number of "bins" for computation of densities

- `ZMAX = <value>`

  Defines the borders of the interval in Z-direction. The densities and the electrostatic potential are calculated in the range of z: $-ZMAX < z < ZMAX$ relative to the membrane middle plane

- `IATE = <list of values>`

  This optional parameter defines which sites to take into account while calculating the densities. Default is all sites. A value equal to 1 says that the given site is accounted while 0 says that this site is not accounted. For example, if the system consists of DMPC lipids (118 atoms in molecule), water (3 atoms), Na and Cl ions, this compute contributions from the lipids only:

  `IATE = 118*1,5*0`

  while this compute contribution from the water oxygen only:

  `IATE = 118*0,1,0,0,0,0`

- `LCHG = <logical>` This parameter tells whether to use a special file containing charges for computation of electron density. In case `LCHG = .true.`, a file defined by parameter `FCHG` is read; it determines contribution of each site into the electron density. If `LCHG = .false.`, contribution of each site is defined from the first letter of the site name (for H, C, N, O, P and S atoms only) and corrected by the partial atom charges.

- `FCHG = <file_name>`

  File name for the case `LCHG = .true.`

- `LSYM = <logical>`

  This parameter tells whether to symmetrize the computed densities relative to the membrane middle plane. Note that even slightest asymmetry in the charge distribution may lead to a potential difference from the both sides of membrane.

7.4 Dielectric constant: diel.f

This utility computes static dielectric permittivity (dielectric constant) from fluctuations of the total molecular momenta using expression (S.W.deLeeuw, J.W.Perram, E.R.Smith, Ann.Rev.Phys.Chem., 37, 245 (1986)):

$$\epsilon = 1 + \frac{4\pi}{3kTV}\langle|\sum_i \vec{\mu}_i|^2\rangle \tag{13}$$

where $\vec{\mu}_i$ is the dipole moment of molecule i and the sum is taken over all the molecules. Note that this formula is applicable only for small and rigid (or

almost rigid) molecules. It is important that the molecules were not broken by the periodic boundary conditions.

This utility needs only a single parameter - the temperature, which should be written directly after the trajectory parameters.

7.5 Diffusion: diffus.f

This procedure computes time dependence of the mean square displacement (MSD) and evaluates the self-diffusion coefficient.

Input parameters for this utility follow after the trajectory parameters in the NAMELIST block `DIFF`:

```
$DIFF
parameter=value(s),
...
$END
```

The following parameters are used:

- `FILDIF = <filename>`

  Defines the name of the output file

- `IDF = <int.num>`

  Defines the type of molecules for which diffusion is calculated

- `DTT = <value>`

  Defines the time interval (in s) for MSD calculations. It is very recommended that this parameter is equal to the time step of the trajectory multiplied by `ISTEP` parameter defined in the trajectory (`TRAJ`) section of the input file. If the above requirement is not fulfilled, the program may still work but less accurately.

- `NTT = <int.value>`

  Defines the number of steps for MSD calculation. The total time of tracking the MSD will be thus `DTT*NTT`.

- `IAT = <int.value>`

  If `IAT=0`, MSD is calculated for the centers of masses of the selected molecules. Otherwise MSD is calculated for `IAT`-th atom of each molecule.

- `LCOM = <logical>`

  Specifies whether to correct for the total center-of-mass motion of the selected molecules (that is, of type `IDF`). If `.true.`, the COM for each molecule is computed relative to center of mass motion of the molecules of this type. The default is `.false.`. Note also, that correction for the center of mass motion of the whole system is not carried out (except the case of only one molecule type and `LCOM=.t.`).

- `FBEG=<value>`

  Defines the beginning of linear fitting of the MSD curve to evaluate the self-diffusion coefficient as: `FBEG*DTT*NTT` . The default value is 0.2 , that is initial 20% of the MSD vs time dependense is not included. It is always recommended to look at the computed MSD vs time dependence to evaluate acceptable value for this parameter.

Futher comments

For each time t the MSD is computed as:

$$\langle \Delta r^2(t) \rangle = \langle (\vec{r}(t_0 + t) - \vec{r}(t_0))^2 \rangle \tag{14}$$

where averaging is taken over all molecules of type `IDF` and all acceptable initial times t_0: $t_{beg} \leq t_0 \leq t_{end} - t$, where t_{beg} and t_{end} are the initial and final time of a *continuous* part of the whole trajectory respectively. A trajectory is regarded as continuous if each next configuration differ from the previous no more than parameter `BREAKM` defined in the trajectory part (`TRAJ`) of the input file.

The output file consists of the following columns:

The first column- time t.

The second: for each t, evaluation of the diffusion coefficient as:

$$D_{av} = \frac{\langle \Delta r^2(t) \rangle}{6t}$$

The third column: for each t, evaluation of the diffusion coefficient as:

$$D_{dif} = \frac{1}{6} \frac{\partial \langle \Delta r^2(t) \rangle}{\partial t}$$

The 4-th column: root square of the MSD (average particle displacement)

5-th – 7-th columns: Evaluation of diffusion coefficient in X-, Y-, and Z-directions as:

$$D_X = \frac{\langle \Delta x^2(t) \rangle}{2t}$$

In all cases, diffusion is given in $10^{-5} cm^2/s$.

7.6 Dryrun: dryrun.f

This utility just reads the trajectories and type the coordinates of the first atom in the output. It can be used as a template to write new analyzing utilities

7.7 Hydrogen bonds: h-bonds.f

This utility identifies hydrogen bonds according to the geometrical criteria and report their statistics.

Input parameters for this utility follow after the trajectory parameters organised in `HBOND` namelist:

```
$HBOND
parameter=value(s),
...
$END
```

Parameters:

- `NHB = <int.value>` Number of different types of hydrogen bonds. Each type of hydrogen bonds is determined by a donor atom (D), acceptor atom (A), and hydrogen atom (H).
- `ACC = <int.value 1>,...,<int.value NHB>` A list of `NHB` iteger numbers representing global site numbers for acceptors for each type of hydrogen bond
- `DON = <int.value 1>,...,<int.value NHB>` A list of `NHB` iteger numbers representing global site numbers for donors for each type of hydrogen bond
- `HYD = <int.value 1>,...,<int.value NHB>` A list of `NHB` iteger numbers representing global site numbers for hydrogen atoms for each type of hydrogen bond
- `RMAX = <value>` Maximum distance between donor and acceptor (in $\mathring{A}$). Default 3.4 $\mathring{A}$.
- `ANGLE = <value>` Maximum angle (in degrees) between D-H and D-A vectors. Default 30 degrees.
- `|FOUT` '¡filename¿'— Name of the output file

7.8 Lateral diffusion: latdiff.f

This utility computes two-dimensional mean square displacement in membrane-like systems and the lateral diffusion coefficients. It works essentially as `diffus` utility with a few exceptions:

- parameter

`IRT = <int.value>`

defines type of "lipid" molecules (that is molecules which build the membrane)

- if parameter

`LCOM = .true.`

is defined, then MSD of lipids are computed relative to the center-of mass motion of *each* monolayer.

- in definition of lateral diffusion, factor 1/4 is used instead of 1/6 in 3D-diffusion.

7.9 Order parameters: order.f

This utility computes the order parameters of selected molecular vectors relative to the Z-axis, as well as angular distributions of these vectors.

Input parameters for this utility follow after the trajectory parameters in the following format:

- All the lines beginning with "#" after the end of trajectory section are considered as commentaries
- The first non-commentary line is the name of the output file
- The second line is an integer number.

  If this parameter is zero (or negative), angular distributions of the specified vectors are computed relative to the positive direction of the Z axis. If this parameter is positive, it defines the type of "lipid" molecules in membrane-like systems. The Z-coordinate of the center of mass of these molecules defines the membrane middle plane, and angular distributions of the molecules which are below the middle plane are calculated relative to the negative direction of the Z-axis.
- The third line is the number of order parameters to compute
- Description of molecular vectors:

  One line per each order parameter (their number is defined above) Each line can be in one of the following form:

  `<n1> <n2> 0`

  Here `<n1>` and `<n2>` are the site numbers (must belong to the same type of molecules) which define the molecular vector. Computation for this vector is done independently from others

  `<n1> <n2> 1 <nref1> <nref2>`

  followed immediately by

  `<n3> <n4>`

  Order parameters and angular distributions for molecular vectors determined by a pair of such lines are computed together. A scalar product of vectors defined by `<n1> <n2>` and `<nref1> <nref2>` is compared with scalar product defined by `<n3> <n4>` and `<nref1> <nref2>`. If the first scalar product is greater, then the vector `<n1> <n2>` is considered as going first and the vector `<n3> <n4>` next; if the second scalar product is greater, then the order in which these vectors follow is changed to opposite (as these lines change places). Note that this comparison is made for each molecule separately. Such arrangement has a sense if, for instance, two hydrogens, equivalent from the force field point of view, are distinguishable in each specific configuration by relation to another, "reference" vector, defined by numbers `<nref1>,<nref2>`, like so-called G1R and G1S order parameters in the G1 glycerol atom of phospholipids.

- Resolution of the orientational distributions

  The last line is an integer. If 0, no orientational distributions are computed. If non-zero, it defined the number of "bins" into which the interval $[-1, 1]$ is divided for calculation of $Cos(\theta)$ distribution.

further comments.
The order parameter for a specific molecular vector is defined as

$$S = \langle \frac{3\cos^2\theta - 1}{2} \rangle$$

where θ is the angle between the molecular vector and Z-axis, and averaging is taken over all molecules and time frames. The output lists also NMR dipole coupling for the corresponding vectors as:

$$C = \langle \frac{3\cos^2\theta - 1}{2|r|^3} \rangle$$

where $|r|$ is the length of the corresponding molecular vector.

7.10 Radial distribution functions: rdf.f

This utility compute radial distribution functions between specified atom pairs.

The input into `rdf` utility consists of a NAMELIST section `RDFIN` and subsequent lines defining sites for RDF calculation. The NAMELIST section

```
$RDFIN
parameter=value(s),
...
$END
```

contains the following parameters

- `FOUTRDF = <filename>`

  The name of the output file

- `NRDF = <int.value>`

  number of RDFs to compute.

- `RDFCUT = <value>`

  Cut-off distance for intermolecular RDF

- `NA = <int.value>`

  Resolution of intermolecular RDF (number of bins within the interval `[0,RDFCUT]`

- `RMI = <value>`
  `RMAX = <value>`

  Boundaries for intramolecular RDFs

- `NAI = <int.value>`

  Resolution of intramolecular RDF (number of bins within the interval `[RMI,RMAX]`

- `LCOMRDF1 = <.true.!.false.> LCOMRDF2 = <.true.!.false.>`

  If .true., compute RDF for the center of mass of the first or/and second molecule, to which the site given in the list of RDFs belong to.

  Default: .false.

Then a number of lines follows defining the sites for RDFs. In this section all the lines beginning with "#" are considered as commentaries. The rules to specify RDFs are the same as in MDynaMix program.

Each (of `NRDF`) RDFs is specified by a single or by several pairs of sites. If RDF is specified by a single pair of sites, two number defining these sites are written in the input. If an RDF needs to be specified by several pair of sites (in order to average them), it is written in the following way:

```
&<num of pairs>
<n1-1>  <n2-1>
<n1-2>  <n2-2>
...

(<num of pairs> times)
```

RDFs between sites `<n1-1> <n2-1>` ... are averaged and counted as a single RDF. For example, for water there exists 3 RDFs (`NRDF=3`) and the list of sites may look like (assuming O as a first atom and hydrogens as 2 and 3):

```
1  1
&2
1  2
1  2
&3
2  2
2  3
3  3
```

7.11 Gyration radius and end-to-end distance: Rg.f

This utility compute distribution of the gyration radius (Rg), and of the distance between two specified atoms in molecule (Ree) of the specified type. Also computed are average values of the gyration radius, root mean square of the distance, and their variances.

The input into `Rg` utility consists of a NAMELIST section `RGLIST`:

```
$RGLIST
parameter=value(s),
...
$END
```

with following parameters:

- `ITYP = <int.value>` Type of the molecule for which computations of Rg and Ree are to be done
- `RMAX = <value>`

  Maximum distance to compute distribution of Rg and Ree
- `NA = <int.value>`

  Resolution to compute distributions of Rg and Ree (number of bins within the interval `[0,RMAX]`
- `IS1 = <int.value>` `IS2 = <int.value>`

  Define site numbers (within the molecule) to compute distance and its distribution
- `FILOUT = 'file_name'`

  Name of the output file with computed data

7.12 Spatial distribution functions: sdf.f

This utility computes spatial distribution functions and creates a file which can be visualized using gOpenMol package (http://.www.csc.fi/gopenmol/).

The input into `sdf` utility consists of a NAMELIST section `SDFIN` and subsequent lines defining sites which will be displayed for visualization of the molecule around which SDF is calculated

The NAMELIST section

```
$SDFIN
parameter=value(s),
...
$END
```

contains the following parameters

- `FILORI = <filename>`

  The name of the output file with SDF written as "formatted" plt-file for gOpenMol package. Before visualization, it must be converted to the binary using "pltfile" utility of gOpenMol (available also from gOpenMol menu).

- FILCRD = <filename> The name of file into which average coordinates of molecule, defining the local basis, are written. These coordinates are computed using the basis of the molecule itself, and they in fact used for visualization of the local basis. This file is written in "XMOL" format. The last column shows the variance (average deviations) of atoms around their average positions in the local coordinate system, which may serve also for evaluation of molecular flexibility.

- NOI = <int.value> Defines the number of equivalent local coordinate systems on the molecule (for example, for DNA SDFs can be defined around each of phosphate group used as a local basis)

- IO1 = <n1_1>,<n1_2>,...,<n1_NOI>
  IO2 = <n2_1>,<n2_2>,...,<n2_NOI>
  IO3 = <n3_1>,<n3_2>,...,<n3_NOI>

  NOI values in each of these three lines. They define NOI local basises. Each basis is defined by three sites (for example, <n1_1>,<n2_1>,<n3_1>. The center of local coordinate system is set at the first site, the X-axis goes as a mediana of 2-1-3 angle, and the Z-axis is perpendicular to the plane defined by these three sites.

- NSOR = <int.value>

  Number of sites for calculation of SDF.

- ISOR = <n1>,<n2> ...

  NSOR integer values. They define SDF of which sites to compute, and then take an average over them.

- RXMAX = <value>
  RYMAX = <value>
  RZMAX = <value>

  These parameters define the box inside which SDF is calculated (for example, from -RXMAX to RXMAX in X-direction)

- NOMX = <value>
  NOMY = <value>
  NOMZ = <value>

  resolution (number of bins) in each direction

After the NAMELIST section, there is a section in the input file which describes which atoms are displayed in the center of SDF as a structure defining the local basis. The first line of this section may be either an integer number or the word "ALL".

If an integer number is specified, it defines the number of atoms displayed in the central structure. Then this number of lines follows, each line per atom, each consisting of NOI + 1 values.

The first position in each line is one of numbers 1,2,3. Positions from 2 to `NOI + 1` define sites which are to be displayed, for each of `NOI` equivalent local coordinate system.

The number in the first position of each line means the following:

1 - coordinates of this atom are averaged independently of the next atom

2 - average coordinates of this atom are calculated taking into account the next atom also (for example, if these are two hydrogens of water)

3 - average coordinates of this atom are calculated taking into account the next two atoms (for instance, treating hydrogens in CH3 group)

Example of these lines for water:

```
1 1
2 2
1 3
```

and for methanol (see the order of atoms in CH3OH.mmol file)

```
1  1
1  2
1  3
3  4
2  5
1  6
```

If parameter `ALL` is specified in the first (and the only) line of this section, then all the atoms of the molecule defining the local basis are displayed, without any special treatment of averaging coordinates of equivalent atoms.

Note about Gopenmol From 2011, there is no longer support for gOpenMol package, though the source and binary executable are still available from the developers.

7.13 Distribution of torsion angles: torsion.f

This utility computes distributions of torsion angles. Several torsion angles can be unified in a group, for which average distribution of the torsion angle is calculated. Distributions for several such groups can be built at a time. For each group, fraction of "gauche" and "trans" conformations is also printed in the output.

Input parameters for this utility follow after the trajectory parameters in the following format:

- All the lines beginning with "#" after the end of trajectory section are considered as commentaries
- The first non-commentary line is the name of the output file
- The second line contains two integer numbers:

  1) number of groups of torsion angles 2) resolution (number of bins)

- The third line (a list of integer numbers) specifies how many torsion angles are in the each group
- In the next lines, site numbers of each torsion are given (four integer numbers for each torsion at each separate line). First, torsions of the first group are listed, then of the second one and so on.

7.14 Ramachandran plot for a pair of torsion angles: torsion2.f

This utility computes two-dimensional distribution for a couple of torsion angles (Ramachandran plot). Distribution of several pairs of torsions can be averaged.

Input parameters for this utility follow after the trajectory parameters in the following format:

- The first non-commentary line is the name of the output file followed by an optional key "tab". If "tab" key specified, the output is given as NxN table of the distribution. Otherwise, the output is given in three columns (as required for e.g. *gnuplot*):

  `<first angle>  <second angle>  <probability>`

- The second line contains two integer numbers:

  1) number of pairs of torsion angles 2) resolution (number of bins)

- Then site numbers of each torsion are given (four integer numbers for each torsion at each separate line). First, torsions of the first pair are listed, then of the second and so on.

7.15 Residence time calculation: restime.f

This utility computes residence time of a molecule (atom) to be within a given distance (R_{hyd}) around another atom (of another molecule). The program computes residence time according to three somewhat different definitions:

1) Algorithm described in paper Impey, R. W.; Madden, P. A.; McDonald, I.R.J. Chem. Phys. 1983, 87, 5071. Average time is evaluated between the moment when a atom come into vicinity of another atom (defined by radius R_{hyd}) until it leave it; however leaving the shell during time less then "permission time" t_{per} (typically 2 ps) not accounted

2) simple average time from the moment when the atom come into vicinity of another atom (that is within sphere of radius $R_{hyd} - \delta r$) to the moment when it leave, that it will be outside the sphere of radius $R_{hyd} + \delta r$

3) from exponential decay of conditional probability P(t) for a atom to remain within a sphere of radius $R_{hyd} + \delta r$ around given atom if it enter the sphere of radius $R_{hyd} - \delta r$ at time t=0 . The exponential decay of $P(t)$ is evaluated at $t < t_{per}$

Input parameters for this utility follow after the trajectory parameters in the NAMELIST block `RESTIM`:

```
$RESTIM
parameter=value(s),
...
$END
```

The following parameters are used:

- `NR1 = <value>`

  number of equivalent sites (first molecule) from which residence time is calculated

- `IR1 = <i_1>, <i_2>,.... <i_NR1>` sites from which residence time is calculated

- `NR2 = <value>`

  number of equivalent sites (second molecule) for which residence time is calculated

- `IR1 = <i_1>, <i_2>,.... <i_NR1>` sites for which residence time is calculated

- `NITT = <value>` number of time intervals for computation of residence time

- `TIMM = <value>` time (in ps) during which probability of leaving hydration shell is calculated

- `TTER = <value>` "permission time", in ps, for criteria 3) of the residence time (default 2ps)

- `RRN = <value>` minimum radius of solvation shell (default 0)

- `RRX = <value>` radius of hydration shell (recommended to set at the corresponding RDF minimum)

- `RRD = <value>` "skin" of solvation shell δr.

  Default: 0.25 Å.

- `FILOUT = <filename>`

  name of the output file

7.16 Time correlation functions: trtcf.f

This utility computes some time correlation functions. For a molecular vector $\vec{n}(t)$, the time correlation function can be determined as:

$$c(t) = \frac{\langle \vec{n}(t_0) \cdot \vec{n}(t_0 + t) \rangle}{\langle \vec{n}(t_0)^2 \rangle} \tag{15}$$

where averaging is taken over all molecules of the given type and all acceptable initial times t_0: $t_{beg} \leq t_0 \leq t_{end} - t$, where t_{beg} and t_{end} are initial and final time of a *continuous* part of the whole trajectory respectively. A trajectory is regarded as continuous if each next configuration differs from the previous by no more than parameter `BREAKM` defined in the trajectory part (`TRAJ`) of the input file.

The program can compute the following time correlation functions:

- Molecular translational velocity (velocity of the center of mass) and their X,Y,Z components in the laboratory or in the molecular coordinate systems
- Molecular angular velocity and their X,Y,Z components in the laboratory or in the molecular coordinate systems
- Dipole moment and its second Legendre polynomial
- Of a chosen molecular vector and its second Legendre polynomial

The second Legendre polynomial TCF is defined as:

$$c(t) = \frac{\langle L_2(\vec{n}(t_0) \cdot \vec{n}(t_0 + t)) \rangle}{\langle L_2(\vec{n}(t_0)^2) \rangle} \tag{16}$$

where $L_2(x) = 0.5(3\cos^2 x - 1)$

Note that compilation of `trtcf.f` requires, in addition to the common header file `tranal.h`, another header file `trtcf.h`, which may need to be edited in order to set array borders matching the studied system

Input parameters for this utility follow after the trajectory parameters in the NAMELIST block `TCF`:

```
$TCF
parameter=value(s),
...
$END
```

The following parameters are used:

- `FILTCF = <filename>`

  Defines the name of the output file

- `NSTEG = <int.num>`

  Number of time steps in the time correlation functions

- `DTCF = <value>`

  TCF time step in fs

  It is recommended that TCF time step be equal to the trajectory time step multiplied by parameter `ISTEP` defined in the trajectory part of the input. The total time of tracking TCF is `NSTEG*DTCF`

- `ITCF = <i1>,<i2>,...,<i12>`

  Flags (integers) specifying which TCF to compute (zero value - do not compute) Meanings of the flags are following:

  - `<i1>` - linear velocity of molecular center of mass
  - `<i2>` - angular molecular velocity
  - `<i3>` - dipole moment
  - `<i4>` - second Legendre polynomial of dipole moment
  - `<i5>` - a specific molecular vector (see below)
  - `<i6>` - second Legendre polynomial of a specific vector
  - `<i7>,<i8>,<i9>` - X, Y, and Z projections of the linear molecular velocity. The projections are calculated in the laboratory frame if the flags are equal to 1, and they are calculated in the principal molecular frame determined by the axis of the inertia tensor if the corresponding flags are equal to 2.
  - `<i10>,<i11>,<i12>` - X, Y, and Z projections of the angular molecular velocity. The projections are calculated in the laboratory frame if the flags are equal to 1, and they are calculated in the principal molecular frame determined by the axis of the inertia tensor if the corresponding flags are equal to 2.

- `N1 = <n1-1>,<n1-2>,...`
  `N2 = <n2-1>,<n2-2>,...`

  `NTYPES` values in each of the two lines, one per each molecular type. These values specify sites (two per each molecule) which define the molecular vector whose TCF is calculated. For each molecular type `m`, sites `<n1-m>` and `<n2-m>` should belong to it.